Search Intelligence: Refresh & Content Opportunity Scoring

Independent Study

Which Pages Should a Content Team Check First?

A careful, honest ranking of pages that are likely losing search visibility. Built and tested from start to finish using real, anonymized Google Search Console data

Shams Sajid Rahman

FlyRank ML Internship: Machine Learning Track

81,383	**0.512**	**0.613**
content items scored	baseline rule AUC	validated model AUC

Repository: github.com/Shams-Sajid-Rahman/Sajid_FlyRank_AI

Built on the FlyRank ML Internship dataset: flyrank.ai

Abstract

A content team has thousands of pages but time to check only a few each week. So which pages should they look at first? To answer this, I built a machine learning model that ranks pages by how likely they are to lose search visibility. The model was trained on two months of anonymized Google Search Console data, using only signals that were known before the outcome happened. The final Random Forest model used prior impressions, position, click through rate, and content age, and reached an AUC of 0.613 when tested on clients it had never seen before. That is a real but small step up from a simple rule I built by hand, which only reached 0.512, close to a random guess. One finding mattered just as much as the model itself. When I tested the model with a normal random split, it looked much stronger, at 0.670 AUC. But once I made sure no client appeared in both the training data and the test data, the honest score dropped to 0.613. This shows that some of the model's apparent strength came from simply remembering client patterns, not from real prediction. The final model is used to build a ranked action list, with three levels of urgency and a reason given for each page. This list is meant to help a human reviewer decide where to look first. It is not a fully automated tool, and it does not claim to prove why any page is declining. Every number in this paper can be checked in the notebooks in the linked repository.

Contents

1 Introduction

Content teams with large search portfolios face a simple problem. They can only review a few pages each week, but their portfolio has thousands of pages. Some pages are quietly losing visibility, some are staying the same, and some are growing. Without a way to sort them, review time gets spent at random, or just on whatever page happens to catch someone's eye.

This tool is built to do one small job well. When given a large set of pages, it creates a short, ranked list of pages worth checking right now. It does not explain why a page is losing visibility, and it does not give an exact fix. It simply narrows down where to look first.

> My first guess was simple. Build a rule by hand that flags pages with a poor position and low visibility. When I tested this rule honestly against real outcomes, it scored an AUC of 0.512, barely better than random guessing. That disappointing result, and the process of finding it, shaped everything that came after.

2 Data

This study uses the FlyRank ML Internship data, hosted on Hugging Face. It joins `fact_content_daily_performance` (Google Search Console metrics) with `dim_content` (content metadata). No client names, domains, or raw URLs appear anywhere in this paper or its outputs. Every identifier is already anonymized as a hash by the dataset itself.

2.1 Window and grain

One row in the dataset stands for one content item, added up over a 30 day window, for one anonymized client. The study covers **February 1 to March 31, 2026**. The first 30 days form the prior window, used for features. The second 30 days form the outcome window, used to define the decline label.

81,383	**25.2%**	**2**
items with enough prior history ($\geq$100 impressions)	of items met the decline definition	months of GSC data used

2.2 What was left out, and why

- **Content with little history**, under 100 impressions in the prior window. A 30 day trend on almost no traffic is mostly noise, not real signal.
- **GA4 engagement fields** for most of the data. Only about 4.2% of March rows had GA4 data at all, too little to use without dropping most of the dataset.
- **Any flags made by FlyRank's own product**, such as an existing health score or triage label. I left these out completely to avoid circular logic, since those flags are already judgments made from the raw data, not raw signals themselves.

This is only one two month window from one point in time. Not every client has the same amount of history tracked. Some clients have far more data than others, so a fixed calendar window does not give every client equal footing. Any seasonal effects specific to February and March would not show up here.

3 Methodology

3.1 Label definition

A content item is labeled **declining** if its impressions in the last 30 days of the window are below 80% of its impressions in the prior 30 days:

$$\texttt{is\_declining} = \texttt{impressions\_last30} < 0.8 \times \texttt{impressions\_prev30}$$

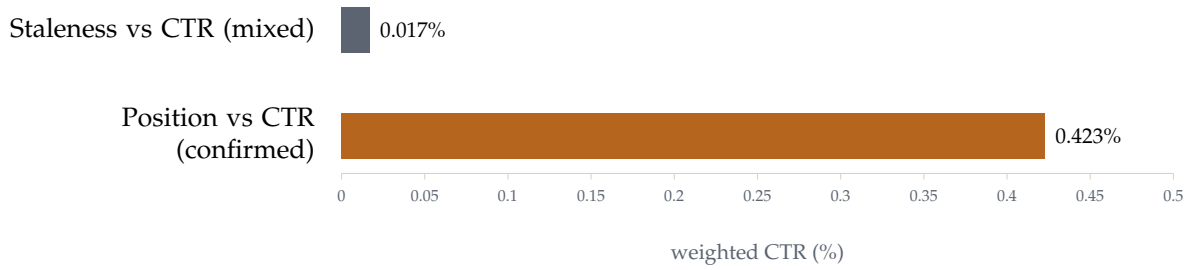
This is a real, directly computed outcome. It is not a proxy borrowed from an existing product label.

3.2 Features known before the outcome window starts

- `imp_prev30`, search impressions from the prior 30 days
- `pos_prev30`, average search position from the prior 30 days
- `ctr_prev30`, click through rate from the prior 30 days
- `days_since_update`, how stale the content is at the start of the outcome window

3.3 Signal check, before building anything

I checked two candidate signals against click through rate before picking a final feature set. This decided which signal the baseline rule and the model would actually lean on.



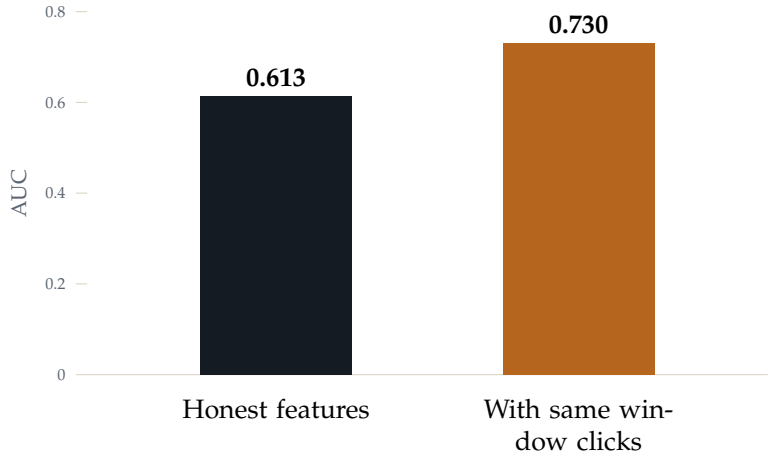
Staleness alone showed a mixed relationship with CTR that did not move in a steady direction. Content left untouched for 90 or more days actually showed a *higher* average CTR than content in the 31 to 90 day range. That is the opposite of what I first assumed. Position, on the other hand, showed a clean, steady relationship. CTR fell step by step, from 0.423% at positions 1 to 3, down to 0.050% at position 21 and beyond.

3.4 Validation design

Content items are grouped by client for the split. Many pages from the same client likely share patterns across the whole site, such as template, niche, or baseline traffic. A plain random split at the row level risks letting the model quietly learn to recognize a client, instead of learning a pattern that works on new clients. So every final result here uses a **split grouped by client** (75% train, 25% test, with zero overlap in clients between the two).

3.5 Leakage check

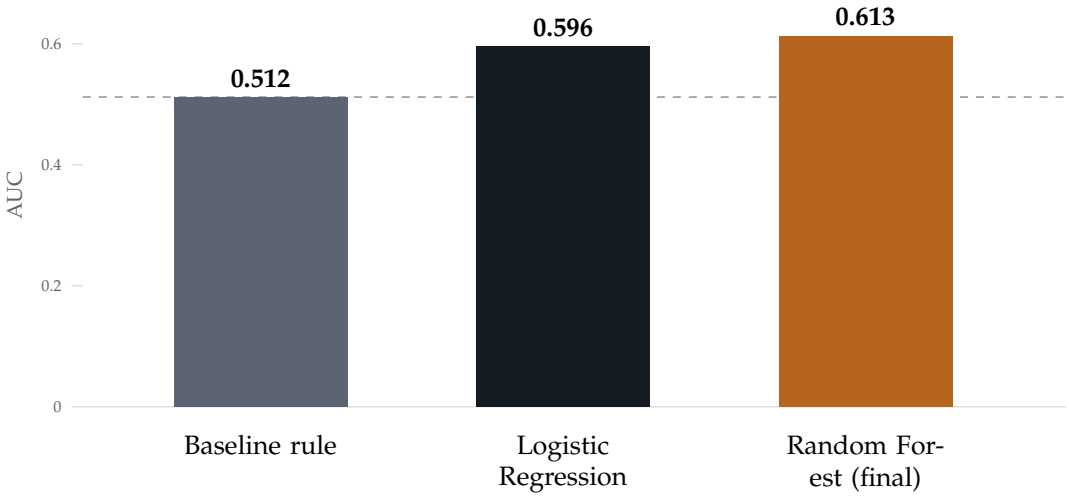
To check that this validation design actually matters, I deliberately reran the model with one leaky feature added. That feature was clicks from the same window as the outcome, not from the prior window.



The jump of plus 0.117 shows the check works as intended. A feature pulled from the same window as the label makes the score look better without adding any real predictive signal. This feature was left out of the final model.

4 Results

Three models were compared on the exact same test split, grouped by client, using the same metric, AUC.



Both models beat the baseline rule, which scored close to random. That alone is a meaningful result, since the baseline rule had been my working assumption going into this study. Random Forest only gave a small gain over Logistic Regression. This suggests most of the improvement comes from using tested features and a real label, not from a more complex model.

4.1 The split design mattered as much as the model

Split design	AUC
Random split, not grouped (naive)	0.670
Grouped by client (honest, reported)	**0.613**

Switching from a random split to a split grouped by client dropped the apparent score by 0.057. That gap likely comes from the model partly recognizing patterns from clients it had already seen in training, instead of truly working on clients it had never seen. Grouping the split by client removes this bias.

> I asked the same grouping question about a published industry paper's model for predicting growth. That paper reports 71% holdout accuracy but never says how the data was split. I could not tell from the paper alone whether its score is inflated the same way mine was. That is exactly the point. Without a stated split design, a headline accuracy number cannot be fully trusted.

4.2 What the model gets wrong

By permutation importance, **click through rate** is by far the strongest predictor, at 0.076, followed by position at 0.041. Staleness and raw impression volume add very little.

	False positives	False negatives
Count	75	6,233
Avg. position	26.1	7.0
Avg. impressions	940	1,917

At the standard 0.5 threshold, the model is very cautious. It is worth noting that the pages it misses tend to have a *better* position and *higher* impressions than its false alarms. Strong current visibility does not reliably protect a page from a coming decline, and right now the model does not weigh that risk enough.

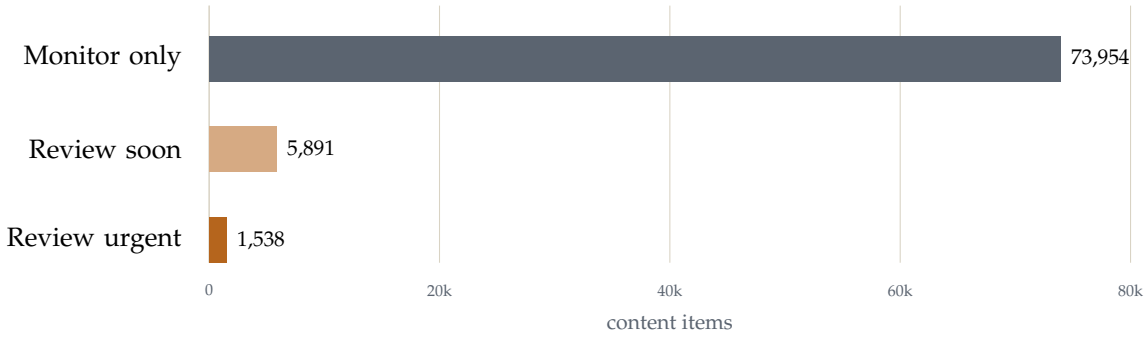
5 Limitations & Honest Framing

Every claim in this paper is kept close to what the data actually supports:

- An AUC of 0.613 is a **real but modest** signal. It is clearly better than chance, but it is not a strong or reliable predictor for any single page.
- All findings here are **observational**. Nothing in this paper proves that refreshing a flagged page will cause it to recover. The model finds pages that look like past declining pages. It does not find the actual cause.
- These results come from one two month window with uneven client history. I have not tested whether they hold up in other seasons, industries, or content types outside this dataset.
- Anywhere the wording could sound like a guarantee, I rewrote it to be careful and directional. I say pages with X are *more likely* to decline, never that pages with X *will* decline.

6 Ranked Recommendations

The tested model scores all 81,383 content items and sorts them into three levels of action. Each item also gets a reason code that explains exactly what was flagged.



Each flagged item gets a reason code: `POOR_POSITION`, `LOW_CTR`, `STALE_CONTENT`, or `GENERAL_RISK`. A reviewer does not just see that a page was flagged. They also see which specific signal triggered it.

6.1 Intended use

This is for a content strategist or SEO reviewer who needs to decide where to spend limited review time on a large portfolio. It is a tool for building a shortlist. It is not an editor, and it does not publish anything on its own.

6.2 What should never be automated

- **No automatic edits.** Content is never rewritten, refreshed, or removed based on the score alone. A person must confirm every action first.
- **No automatic client communication.** A finding is never sent to a client without a human reviewing it first.
- **No performance judgments.** Scores are never used to judge a writer or a team. The signal is too weak, at AUC 0.613, to carry that weight fairly.

6.3 Monitoring and retrain triggers

The pipeline should be run again at least once every 60 to 90 days. It should also be rerun whenever a large number of new clients or content items are added, or if the honest, tested AUC drops well below about 0.55 on a fresh run. A drop like that would be a sign the underlying pattern is weakening and needs a real review, not just a routine refresh.

7 Reproducibility

Every number in this paper can be traced back to a notebook that was committed and run in the project repository:

- `work/notebooks/w02_ml_task_framing.ipynb`, problem framing and task type
- `work/notebooks/w03_data_contract.ipynb`, data grain, verification queries, feature and label definitions, first leakage check
- `work/notebooks/w04_baseline_score.ipynb`, signal checks, baseline rule

- `work/notebooks/w05_model.ipynb`, model training, grouped split, model vs. baseline
- `work/notebooks/w06_validation_audit.ipynb`, before and after split comparison, leakage audit, claim rewrite
- `work/notebooks/w07_action_playbook.ipynb`, final scoring, action queue, exports

Metrics referenced in this paper are committed at `work/outputs/w07_metrics.json`. Repository: https://github.com/Shams-Sajid-Rahman/Sajid_FlyRank_AI.

8 Acknowledgments & Data Credit

This study is built entirely on the **FlyRank ML Internship dataset**. It is an anonymized Google Search Console and content metadata warehouse made available for this internship track. All of the performance data, client identifiers (already anonymized), and content metadata come from that release. No data was collected on my own for this study.

Built on the FlyRank ML Internship dataset: flyrank.ai

Thanks also to the FlyRank ML Internship program for the clear, week by week track that this capstone was built on. Each stage of this paper, the data contract, the baseline, the model, the validation audit, and the action playbook, matches a specific weekly assignment notebook. All of these notebooks are committed in the linked repository.